Runbook: operating and troubleshooting ragcap

For whoever runs the system day to day on the Apple M5 Pro. Assumes the SOP has been completed. All commands run in the build folder.

1. Components and where they live

Component	Where	Notes
Postgres 16.2 + pgvector 0.6.2	pgctl.sh; data in .pgdata/; 127.0.0.1:5433, TCP only	native from the pip wheel pgserver; does not start at login: ./services.sh start
Ollama 0.40.9	~/.local/opt/ollama (command ~/.local/bin/ollama); models in ~/.ollama/models; localhost:11434	does not start at login: ./services.sh start; log ~/.ollama/serve.log
Models	llama3.2:3b, nomic-embed-text (required); qwen3:8b (recommended grader); qwen2.5vl:3b (drawings)	override with LLM_MODEL, EMBED_MODEL, GRADE_*, VISION_MODEL in .env
Credentials	.env (mode 600, gitignored)	never commit; never paste into chat or documents
Code	ragcap/ (config, db, llm, ingest, graph, vision, tracing), api.py, ui.py, try.py	git repository in this folder
Scripts	services.sh, pgctl.sh, recreate.sh, scenarios.py, eval.py, run_series.sh, b1_check.py, b1_eval.py, pid_eval.py	see section 3
Traces and results	traces.jsonl (one line per question, gitignored); results/	m5_summary.md (lean), qwen3esc_summary.md (recommended), scenarios_*.md

2. Start, stop, health

```
./services.sh start            # Ollama (ollama serve in the background) + Postgres; prints status
./services.sh status           # read-only: Ollama version and models in memory, Postgres counts, busy ports
./services.sh check            # clean-start check (2a): PASS/FAIL per item, CLEAN START OK or exit 1
./services.sh stop             # Postgres only (data kept in .pgdata/); --all also stops Ollama
.venv/bin/uvicorn api:app --port 8000 &   # web page + API; curl -s localhost:8000/health (8000 taken? --port 8765)
```

/health returns the document count, chunk count and uploaded tables: 16, 16, [broker_inventory] at the baseline.

2a. Checking for a clean start

Run after a reboot, after the Mac slept, after moving the folder, and before any evaluation series. It takes about 10 seconds and changes nothing except a temporary API on port 8765 and three lines in traces.jsonl.

#	Do	Expect	If not
1	./services.sh status	ollama up, postgres up	./services.sh start
2	./services.sh check	every line PASS (---- lines are information) and CLEAN START OK; echo $? prints 0	fix the first FAIL; the line says how
3	read the data line	16 docs, 16 chunks, 8 superseded, 1 tables	uploads or drawings were added: expected after use; ./recreate.sh restores the baseline before measuring
4	read the smoke lines	general, database ('2' brokers), documents ('75'%) PASS, each under ~2 s (the first is slower while models load)	section 7 stage probe
5	before a series	caffeinate -i ./run_series.sh <label>; it refuses to start if a service is down	section 2b

2b. After a reboot or a sleep

Neither the database nor the standalone Ollama starts on its own. ./services.sh start && ./services.sh check brings both back and proves the path works. A sleep pauses running work: on 2026-10-22 an eval.py started without caffeinate stood still for 16 minutes (pmset -g log showed 'Entering Sleep state due to Idle Sleep') and finished after the wake. Run long commands under caffeinate -i; run_series.sh, run_answer_ablation.sh and recreate.sh do it themselves.

3. Routine tasks

Task	Command
Ask a question from the shell	.venv/bin/python try.py "question"
Web page / Gradio	.venv/bin/uvicorn api:app --port 8000 then http://127.0.0.1:8000/ ; .venv/bin/python ui.py then http://127.0.0.1:7860
Add documents	.venv/bin/python -m ragcap.ingest <file-or-folder> [--version N] (same name and version replaces it)
Add a CSV/Excel table	.venv/bin/python -m ragcap.ingest <file.csv> (table name from the file name; Excel: one table per sheet)
Add a drawing (P&ID)	.venv/bin/python -m ragcap.ingest <file.png> (5-12 s per drawing with qwen2.5vl:3b)
Add a new version of a document	give it front-matter (doc, version, effective_date, superseded) and ingest; mark the old row: UPDATE documents SET superseded=true WHERE name=... AND version<N; then b1_check.py --no-trap
Version filter	on by default; VERSION_AWARE=0 only for the ablation; never set 0 in .env
Use the recommended grader	add GRADE_MODEL=qwen3:8b, GRADE_MODE=escalate, GRADE_MAX_TOKENS=600 to .env; restart the web page or Gradio
Run the scenario catalogue	.venv/bin/python scenarios.py [--with-drawings] (about 1 minute; exit 1 only on an unexpected failure)
Run an evaluation	caffeinate -i .venv/bin/python eval.py [--golden golden/golden_heldout.jsonl] [--tag name] (under 1 minute)
Run a series	./run_series.sh <label> <model>; .venv/bin/python summarize_series.py <label> (about 3-7 minutes)
Re-prove the version filter	.venv/bin/python b1_eval.py --latency 3 (results/ablation_summary.md)
Re-run the P&ID evaluation	copy results/pid_eval_summary.md first, then .venv/bin/python pid_eval.py [--ingest]
Rebuild to the baseline	./recreate.sh (RECREATE OK; resets the database to the 16 documents)
Models in memory / free them	./services.sh status or ollama ps; ollama stop <model>; idle models unload after 5 minutes
Inspect recent traces	tail -n 3 traces.jsonl | .venv/bin/python -m json.tool
List what is stored	./pgctl.sh psql -c "select d.name,d.version,count(c.id) from documents d join chunks c on c.doc_id=d.id group by 1,2"

4. Backup, restore, reset

```
./pgctl.sh dump > ragcap_$(date +%Y%m%d).dump      # backup before re-ingesting or changing the embedding model
./pgctl.sh restore < ragcap_YYYYMMDD.dump           # restore (verified: all 16 chunks back)
./pgctl.sh reset                                    # DESTROYS .pgdata/; then ./recreate.sh
```

Dump files contain document text: keep them out of git. Take a dump before any reset.

5. Troubleshooting

Symptom	Likely cause	Fix
services.sh check: FAIL in the first lines, smoke tests skipped	a service is down, or the venv or .env is missing	the FAIL line names the fix; usually ./services.sh start
Connection refused on 5433	the database is stopped	./pgctl.sh start; details in .pgdata/server.log
Connection refused on 11434	Ollama is stopped	./services.sh start
password authentication failed	.env changed after the database was created	restore the old .env, or ./pgctl.sh reset and ./recreate.sh
pg_ctl: could not start server	port 5433 in use, or a stale lock after a crash	lsof -nP -iTCP:5433; read .pgdata/server.log
address already in use (web page or Gradio)	another program owns the port	--port 8765, or GRADIO_SERVER_PORT=7861 .venv/bin/python ui.py
A run stood still, then finished much later	the Mac slept	caffeinate -i; section 2b
Everything answers 'not in the knowledge base'	nothing ingested, or the grader rejects everything	check /health; stage probe (section 7); try the recommended grader
Right passage retrieved but 'not in the knowledge base'	the 3B grader rejected it	recommended grader; stage probe
Answers quote a retired value	VERSION_AWARE is off, or the old row is not marked superseded	./pgctl.sh psql -c "select name,version,superseded from documents"; b1_check.py
Same question gives a different SQL query	3B model variability	read the sql field in traces.jsonl; judge changes over 3 runs
'only a single read-only SELECT is allowed'	the model wrote a write statement	expected safety behaviour
AssertionError: EMBED_DIM does not match	the embedding model changed	change EMBED_DIM and vector(N) together, recreate the chunks table, re-ingest
Image ingest fails	vision model not pulled	ollama pull qwen2.5vl:3b
Memory pressure	several models resident	ollama ps; ollama stop <model>; section 6
git refuses to run (Xcode license)	license not accepted	sudo xcodebuild -license accept (admin)
First request after idle takes several seconds	Ollama loading the model	expected; later requests are fast

6. Models and memory

Profile	Models	Disk	Memory	Unseen / tuned answers
Lean (default)	llama3.2:3b + nomic-embed-text	2.3 GB	~2.7 GB	80% / 92.6%
Recommended	Lean + qwen3:8b grader	7.5 GB	~2.7 GB, peak ~8.2 GB	96% / 92.6%
With drawings	Recommended + qwen2.5vl:3b	10.7 GB	+5.0 GB while ingesting	P&ID 38/39 tags

```
ollama list                    # installed models and disk size
ollama ps                      # models in memory now (under-reports vision models)
ollama stop qwen3:8b           # free memory now
ollama rm <model>              # free disk; ollama pull <model> restores it
```

7. Debugging a wrong answer, stage by stage

```
.venv/bin/python - <<'EOF'
import ragcap.graph as g
s = g.router({"question": "YOUR QUESTION", "history": [], "trace": []})
print(s["route"], "|", s["standalone"])          # 1. route and standalone question
```

```
r = g.retrieve({**s}); print([(c["doc"], c["dist"]) for c in r["chunks"]])   # 2. is the right document in the top 6?
gr = g.grade({**s, **r, "trace": []}); print([c["doc"] for c in gr["relevant"]])  # 3. did grade keep it?
EOF
```

The first stage that goes wrong is the one to fix. A retrieval problem shows as the wrong document or a high distance; a grading problem as an empty relevant list while the right document was retrieved; an answer problem as the right document kept and a wrong answer.

8. Security notes

- The database and Ollama listen on localhost only; the database has no Unix socket. - The SQL agent runs one SELECT/WITH statement in a READ ONLY transaction with a 5 s timeout, but as the user that owns the tables; beyond a local demo give it a separate read-only role. - /upload has no authentication or size limit; keep the API on localhost. - Uploaded text goes to the model unfiltered, so a malicious document can attempt prompt injection.

9. Evaluation pass/fail guide (M5 baselines)

Metric	Lean baseline	Recommended baseline	Accept a change if
Router accuracy, tuned / unseen	100% / 100%	100% / 100%	no drop
Answers correct, tuned / unseen	92.6% / 80.0%	92.6% / 96.0%	no drop on either set, over 3 runs
Retrieval hit rate	100%	100%	>= 95%
Unanswerable refused	4/4 (both sets)	4/4	never regress
Scenario catalogue	12 pass, 0 unexpected	-	0 unexpected failures
Superseded chunks, filter ON	0/192	0/192	stays 0

Judge a change on the failing case ids, not only the percentage. Never edit golden/golden.jsonl or golden_heldout.jsonl to make a run pass.

10. Rebuilding elsewhere

docs/INSTALL_AND_USE.md (PDF INSTALL_ragcap.pdf) installs everything by hand; docs/RECREATE.md (PDF RECREATE_ragcap.pdf) is the from-zero guide with the scenario catalogue and the fingerprints; ./recreate.sh runs it unattended and exits 1 on any mismatch. Run it after any change to the corpus, the schema or retrieve().

11. The showcase site

site/ is static: results, the flow, screenshots, the documents, and a replay of recorded runs (site/data/replay.json, rebuilt from results/ by site/build_replay.py). It is updated locally; publishing (./publish_site.sh, which pushes the gh-pages branch) and Vercel wait for the deployment stage. Never put the database password, traces or dumps in site/.

12. Change control for prompt or pipeline changes

#	Action	Command / rule
1	Reproduce the failure on one question	.venv/bin/python try.py "question" ; then the stage probe (section 7)
2	Find the first stage that goes wrong	router -> retrieve -> grade -> generate (or sql_agent -> generate_sql)
3	Change ONE prompt or rule	edit ragcap/graph.py; never paste golden questions into prompts
4	Probe the question plus 3 neighbours	.venv/bin/python try.py ... (watch for the fix leaking into other queries)
5	Commit before measuring	git add -A && git commit -m "what and why" (git needs the Xcode license accepted)
6	Measure	./run_series.sh <label> llama3.2:3b; .venv/bin/python scenarios.py

#	Action	Command / rule
7	Compare by failing ids	results/<label>_summary.md against results/m5_summary.md
8	Record it	add the result to the SOP, then python docs/build_pdfs.py (needs reportlab)